COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	September 1, 2026	**DATE SUBMITTED:**	September 1, 2026
NAMES:	Ramos, Samuel Reyes, Fenzie Aaron A. Rodulfa, Jose Martin C. Salcedo, Lance Santiago, Nina Denise M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 7.1 USING SORTING ALGORITHMS

1. PROCEDURE OUTPUT

ANSWER THE FOLLOWING QUESTIONS:

1. **What is a sorting algorithm?**

- A sorting algorithm is a method used to arrange data in a specific order. That way, it can arrange numbers, names, or other data from smallest to largest or alphabetically. For instance, if we have {5, 2, 8, 1, 21}, using bubble sort, we can arrange them into {1, 2, 5, 8, 21}.

2. **Where can sorting algorithms be used?**

- Sorting algorithms can be used in many ways whenever data needs to be organized. Furthermore, this concept is commonly used for arranging numbers, names, grades, pieces, and other information. For example, it can be used to arrange files by name, organize contacts alphabetically, or rank players based on their scores or grades.

3. **Explain the different types of sorting algorithms.**

- If we want to use Comparison-Based Sorting Algorithms by comparing each variable with each other we could use Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, Cycle Sort, and 3-way Merge Sort. On the other hand, we have Non-Comparison Based Sorting in which it does not directly compare elements. If we are just aiming to use properties such as the values or digits of the elements to sort, then we can proceed to this one, examples include: Counting Sort, Radix Sort, Bucket Sort, Pigeonhole Sort. Furthermore, we also have Hybrid Sorting Algorithms that combine two or more sorting methods in order to make the process more efficient. Meaning to say, it is a combined type of sorting algorithm, examples include IntroSort and TimSort (GeeksforGeeks, 2026).

4. Give sample programs in C++ that use sorting algorithms, specifically selection sort, insertion sort, and bubble sort. Explain how the programs work.

BUBBLE SORT:

Python | Java | C | C++

```cpp
// Bubble sort in C++

#include <iostream>
using namespace std;

// perform bubble sort
void bubbleSort(int array[], int size) {

  // loop to access each array element
  for (int step = 0; step < size -1; ++step) {

    // loop to compare array elements
    for (int i = 0; i < size - step - 1; ++i) {

      // compare two adjacent elements
      // change > to < to sort in descending order
      if (array[i] > array[i + 1]) {

        // swapping elements if elements
        // are not in the intended order
        int temp = array[i];
        array[i] = array[i + 1];
        array[i + 1] = temp;
      }
    }
  }
}

// print array
```

```cpp
// print array
void printArray(int array[], int size) {
  for (int i = 0; i < size; ++i) {
    cout << "  " << array[i];
  }
  cout << "\n";
}

int main() {
  int data[] = {-2, 45, 0, 11, -9};

  // find array's length
  int size = sizeof(data) / sizeof(data[0]);

  bubbleSort(data, size);

  cout << "Sorted Array in Ascending Order:\n";
  printArray(data, size);
}
```

```
Output

Sorted Array in Ascending Order:
  -9  -2  0  11  45


=== Code Execution Successful ===
```

Example taken from Programiz.com

- In bubble sorting, it compares adjacent elements and swaps them if they are in the wrong order. Basically, if we visualize bubble sorting, the first element will be compared to the second element, then the second to the third, and so on until the end of the array. This process is repeated until all the elements are arranged in accordance to their order.

INSERTION SORT

Insertion Sort in Python, Java, and C/C++

Python | Java | C | C++

```cpp
// Insertion sort in C++

#include <iostream>
using namespace std;

// Function to print an array
void printArray(int array[], int size) {
  for (int i = 0; i < size; i++) {
    cout << array[i] << " ";
  }
  cout << endl;
}

void insertionSort(int array[], int size) {
  for (int step = 1; step < size; step++) {
    int key = array[step];
    int j = step - 1;

    // Compare key with each element on the left of it until an element smaller than
    // it is found.
    // For descending order, change key<array[j] to key>array[j].
    while (j >=0 && key < array[j]) {
      array[j + 1] = array[j];
      --j;
    }
    array[j + 1] = key;
  }
}
```

```
// Driver code
int main() {
  int data[] = {9, 5, 1, 4, 3};
  int size = sizeof(data) / sizeof(data[0]);
  insertionSort(data, size);
  cout << "Sorted array in ascending order:\n";
  printArray(data, size);
}
```

```
Output

Sorted array in ascending order:
1 3 4 5 9


=== Code Execution Successful ===
```

Example taken from Programiz.com

- Insertion Sort takes one element at a time and inserts it into its proper position among the already sorted elements. For instance, if we want to sort these elements {4, 1, 7, 3, 8, 5, 2, 6} from lowest to highest, this method would pick one element at a time. For example, it first gets element 1, then it will compare itself to the already sorted element 4. Since 1 is smaller than 4, it would move 4 to the right and place 1 before it, which would make it {1, 4, 7, 3, 8, 5, 2, 6}. The method then will repeat this process until the correct order has been made.

SELECTION SORT

Python | Java | C | C++

```cpp
// Selection sort in C++

#include <iostream>
using namespace std;

// function to swap the the position of two elements
void swap(int *a, int *b) {
  int temp = *a;
  *a = *b;
  *b = temp;
}

// function to print an array
void printArray(int array[], int size) {
  for (int i = 0; i < size; i++) {
    cout << array[i] << " ";
  }
  cout << endl;
}

void selectionSort(int array[], int size) {
  for (int step = 0; step < size - 1; step++) {
    int min_idx = step;
    for (int i = step + 1; i < size; i++) {

      // To sort in descending order, change > to < in this line.
      // Select the minimum element in each loop.
      if (array[i] < array[min_idx])
        min_idx = i;
    }

    // put min at the correct position
    swap(&array[min_idx], &array[step]);
  }
}

// driver code
int main() {
  int data[] = {20, 12, 10, 15, 2};
  int size = sizeof(data) / sizeof(data[0]);
  selectionSort(data, size);
  cout << "Sorted array in Acsending Order:\n";
  printArray(data, size);
}
```

Output

```
Sorted array in Acsending Order:
2 10 12 15 20


=== Code Execution Successful ===
```

Example taken from Programiz.com

- In Selection Sort, it first finds the smallest element from the unsorted part of the array and places it in the correct order. For example, in the above case, we have {20, 12, 10, 15, 2}, it first finds 2 as the smallest element and swaps it with 20, making {2, 12, 10, 15, 20}. This method continues its process with the remaining elements until the array is sorted from lowest to highest.

REFERENCES (APA FORMAT)

GeeksforGeeks. (2026, January 20). *Sorting algorithms.* https://www.geeksforgeeks.org/dsa/sorting-algorithms/

Programiz. (n.d.). *Bubble sort.* https://www.programiz.com/dsa/bubble-sort

Programiz. (n.d.). *Insertion sort.* https://www.programiz.com/dsa/insertion-sort

Programiz. (n.d.). *Selection sort.* https://www.programiz.com/dsa/selection-sort

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

N/A

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: N/A

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: N/A

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: N/A

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.